LambdaScript : A Functional Programming Language

Alex Kozik
Cornell University
Ithaca, NY
ajk333@cornell.edu

***Abstract*—We present LambdaScript, a statically-typed functional programming language featuring Hindley-Milner type inference, polymorphic algebraic data types, and comprehensive pattern matching. LambdaScript combines the expressiveness of languages like OCaml and Haskell with a clean syntax and rigorous formal semantics. The language supports advanced features including mutually recursive types, exhaustiveness checking for pattern matching, and first-class functions with closures. We describe the formal type system, operational semantics, and implementation of a complete interpreter with over 700 unit tests demonstrating correctness.**

***Index Terms*—functional programming, type inference, algebraic data types, pattern matching, programming languages**

I. Introduction

Functional programming languages offer powerful abstractions for reasoning about programs through immutability, higher-order functions, and strong type systems. LambdaScript is designed to explore these concepts with a focus on:

- **Type Safety**: Static type checking with automatic type inference eliminates entire classes of runtime errors
- **Expressiveness**: Algebraic data types and pattern matching enable concise, readable code
- **Formal Foundations**: Rigorous semantics based on type theory and lambda calculus
- **Practical Features**: List comprehensions, operators as values, and comprehensive built-in functions

This paper presents both the theoretical foundations and practical implementation of LambdaScript, demonstrating how formal semantics can guide language design and implementation.

II. Language Overview

A. Core Features

LambdaScript provides a rich set of features for functional programming:

1) Type System:

- Basic types: `int`, `float`, `bool`, `string`, `char`, `unit`
- Composite types: functions (`a -> b`), lists (`[a]`), tuples (`(a, b, c)`)
- Polymorphic types with type variables (`'a`, `'b`)
- Algebraic data types with constructors
- Recursive and mutually recursive types

2) Expressions: The language supports standard functional programming constructs including lambda expressions, let bindings, conditionals, pattern matching via case expressions, and list operations including comprehensions.

3) Example Program: A simple example demonstrating type inference and pattern matching:

```
let rec length = fn lst ->
  case lst do
  | [] -> 0
  | h :: t -> 1 + length t

let result = length [1, 2, 3, 4, 5]
(* result : int = 5 *)
```

III. Formal Semantics

A. Syntax

We define the abstract syntax of LambdaScript using BNF grammar. Expressions include literals, variables, lambda abstractions, applications, let bindings, conditionals, and pattern matching.

B. Type System

1) Type Inference Algorithm: LambdaScript implements constraint-based type inference based on the Hindley-Milner algorithm. The process consists of three phases:

1) **Constraint Generation**: Traverse the abstract syntax tree and generate type equations
2) **Constraint Solving**: Unify type equations using a substitution-based algorithm
3) **Generalization**: Introduce universal quantifiers for polymorphic types

2) Type Relations: We define a type relation of the form:

$$\Gamma \vdash e : \tau \Rightarrow C$$

This states that under type environment Γ, expression e has type τ and produces constraint set C.

3) Polymorphism: LambdaScript supports let-polymorphism, allowing polymorphic generalization at let-bindings but not lambda abstractions, consistent with ML-family languages.

C. Operational Semantics

1) Small-Step Semantics: We define the evaluation relation:

$$\langle e, \rho \rangle \to \langle e', \rho' \rangle$$

where ρ is the dynamic environment mapping variables to values.

2) Values and Closures: Functions are represented as closures capturing their defining environment, enabling lexical scoping and higher-order functions.

D. Algebraic Data Types

1) Type Definitions: Sum types are defined with the syntax:

```
type Option<'a> =
  | None
  | Some of 'a
```

2) Recursive Types: Recursive types use fixed-point semantics with isorecursive representation:

```
type rec List<'a> =
  | Nil
  | Cons of ('a, List<'a>)
```

3) Mutually Recursive Types: Multiple types can reference each other using the `and` keyword:

```
type rec Tree<'a> =
  | Leaf of 'a
  | Node of ('a, Forest<'a>)
and Forest<'a> =
  | Empty
  | Trees of (Tree<'a>, Forest<'a>)
```

E. Pattern Matching

1) Pattern Syntax: Patterns include literals, variables, wildcards, constructors, tuples, and list patterns (cons and nil).

2) Exhaustiveness Checking: LambdaScript implements exhaustiveness checking for pattern matching using a matrix-based algorithm. This ensures all possible cases are handled at compile time, preventing runtime match failures.

The algorithm:

1) Represents patterns as a matrix where rows are match branches
2) Specializes the matrix for each constructor
3) Recursively checks coverage for all constructors
4) Reports missing patterns as compile-time warnings

IV. Implementation

A. Architecture Overview

The LambdaScript interpreter consists of five main components:

1) **Lexer**: Tokenizes source code into lexical tokens
2) **Parser**: Constructs abstract syntax trees using parser combinators
3) **Type Checker**: Performs constraint-based type inference
4) **Evaluator**: Executes programs using environment-based interpretation
5) **REPL**: Provides an interactive programming environment

B. Lexical Analysis

The lexer recognizes keywords, operators, identifiers, literals, and comments. Special handling ensures operators can be used as first-class values when parenthesized (e.g., `(+)`).

C. Parsing

The parser is implemented using parser combinators, enabling compositional grammar definitions. Each syntactic construct has a corresponding parser function that can be combined to build complex parsers.

D. Type Checking Implementation

1) Constraint Generation: The type checker traverses the AST recursively, generating fresh type variables and constraint equations. Each expression form has specific typing rules that produce constraints.

2) Unification: Type equations are solved using Robinson's unification algorithm. The unifier computes most general substitutions, handling:

- Variable-to-type bindings
- Function type decomposition
- List and tuple type decomposition
- Constructor type application
- Occurs check to prevent infinite types

3) Generalization and Instantiation: Polymorphic types are generalized by quantifying over free type variables not bound in the environment. Instantiation creates fresh type variables for each use of a polymorphic binding.

E. Evaluation

1) Environment Model: The evaluator maintains an environment as an association list mapping identifiers to values. Closures capture their defining environment, enabling proper scoping.

2) Recursive Functions: Recursive bindings are handled using cyclic data structures with OCaml references. The function closure's environment is backpatched to include the function itself.

3) Mutually Recursive Definitions: Multiple mutually recursive functions are defined together, with all names added to the environment before evaluating any function bodies.

F. Built-in Functions

LambdaScript provides essential built-in functions including:

- I/O: `print`, `println`
- Conversions: `int_to_str`, `int_to_float`, `string_to_list`
- Higher-order: `map`, `filter`, `reduce_left`, `reduce_right`

G. Advanced Features

1) List Comprehensions: List comprehensions provide concise syntax for creating lists:

```
[x * x | x => [1...10], x % 2 == 0]
(* Result: [4, 16, 36, 64, 100] *)
```

2) Operators as First-Class Values: Binary operators can be used as functions by parenthesizing them:

```
let add = (+) in
map add [1, 2, 3] [4, 5, 6]
```

V. Case Study: Red-Black Trees

To demonstrate LambdaScript's expressiveness, we implement balanced red-black trees. This showcases:

- Algebraic data types with color tagging
- Nested pattern matching for tree rotations
- Polymorphic recursive data structures
- Purely functional algorithms

The implementation includes insertion with automatic balancing, search operations, and tree property queries (size, height, minimum, maximum). All operations maintain red-black tree invariants through pattern matching alone.

VI. Evaluation and Testing

A. Test Suite

LambdaScript includes over 700 unit tests covering:

- Type checking and inference (200+ tests)
- Expression evaluation (250+ tests)
- Pattern matching and ADTs (150+ tests)
- Higher-order functions (100+ tests)
- Edge cases and error conditions (50+ tests)

B. Code Coverage

Using Bisect_ppx, the test suite achieves comprehensive coverage of the interpreter codebase, validating both normal operation and error handling paths.

C. Performance Characteristics

While LambdaScript prioritizes correctness and clarity over performance, the interpreter achieves reasonable execution speeds for educational and prototyping purposes. Key operations maintain expected algorithmic complexities.

VII. Related Work

LambdaScript draws inspiration from several influential functional languages:

- **ML/OCaml**: Type inference algorithm, module structure, syntax
- **Haskell**: Purity emphasis, list comprehensions, type classes
- **Scheme**: Simple core with powerful abstractions

Our contribution is a complete, formally specified language with both theoretical foundations and working implementation, suitable for teaching and research.

VIII. Future Work

Potential extensions to LambdaScript include:

- Type classes for ad-hoc polymorphism
- Module system for code organization
- Effect system for tracking side effects
- Compilation to JavaScript or native code
- IDE support with language server protocol
- More sophisticated optimization passes

IX. Conclusion

LambdaScript demonstrates that rigorous formal semantics and practical implementation can coexist harmoniously. The language provides a solid foundation for exploring functional programming concepts while maintaining accessibility through clean syntax and comprehensive tooling. With extensive testing and clear documentation, LambdaScript serves as both a teaching tool and a research platform for programming language concepts.

The complete implementation is available as open source, including formal semantics documentation, full test suite, and example programs demonstrating language features.

Acknowledgment

Thanks to the OCaml community for excellent tools and libraries that made this implementation possible.

References

[1] Robin Milner, "A Theory of Type Polymorphism in Programming," Journal of Computer and System Sciences, vol. 17, no. 3, pp. 348–375, 1978.

[2] Luis Damas and Robin Milner, "Principal type-schemes for functional programs," Proceedings of POPL '82, pp. 207–212, 1982.

[3] Benjamin C. Pierce, Types and Programming Languages, MIT Press, 2002.

[4] Chris Okasaki, Purely Functional Data Structures, Cambridge University Press, 1998.

[5] Simon Peyton Jones, The Implementation of Functional Programming Languages, Prentice Hall, 1987.